Relatório – Tarefa 2: Grafo de palavras de 4 letras

Disciplina: TEG (2026/2) · Linguagem: C (C11) · Ambiente: Ubuntu

1. Identificação da equipe

- Integrante 1: Nicolas Oliveira Veiga – Matrícula: 206101254
- Integrante 2: Guilherme Wippel Kormann – Matrícula: [preencher]
- Ciência da Computação / CCI251-04U / professor: Gilmário Barbosa
- Repositório no GitHub: https://github.com/nic-veiga/trabalho-teg.git
- Arquivo de entrega: NicolasV_GuilhermeW.zip

2. Identificação da tarefa

Tarefa 2 – Implementação de um grafo com carga primária a partir de um arquivo de base.

O objetivo é construir um grafo não direcionado e ponderado G(V,E) em que os vértices são palavras em português de 4 letras, lidas de uma base fornecida no Moodle (`entrada.txt`), e em que dois vértices p e q são ligados por uma aresta quando as palavras diferem em **exatamente uma posição**. A ideia é estudar a transformação de uma palavra em outra, trocando uma letra por vez. Exemplo do enunciado: LAGO → LOGO → LEGO → CEGO → CEGA → CENA.

Requisitos do enunciado e onde cada um foi atendido:

- **(a) Implementação em C:** C11, usando somente a biblioteca padrão.
- **(b) Carga primária do grafo a partir da base:** módulo `grafo.c`, função `grafo_carregar`.
- **(c-i) Componentes conexos, tamanhos, palavra central (grau máximo) e de menor conexão (grau mínimo):** módulo `analise.c`, funções `analisar_componentes` e `imprimir_componentes`.
- **(c-ii) Grafo simples ou multigrafo, com contagem de laços e arestas múltiplas:** módulo `analise.c`, função `verificar_tipo_grafo`.
- **(c-iii) Sequência de arestas entre duas palavras fornecidas pelo usuário, com Dijkstra, e comparação:** módulo `dijkstra.c` e função `consultar` em `main.c`.

3. Explicação conceitual da solução

3.1 Entrada

- **Arquivo de base (`entrada.txt`):** uma palavra por linha. O arquivo fornecido tem 1604 linhas, todas com 4 letras ASCII (sem acentos), finais de linha no padrão Windows (CRLF) e sem quebra de linha após a última palavra.
- **Linha de comando:** `./teg [opções] [arquivo [origem destino]]`. Sem argumentos, o programa usa `entrada.txt`.
- **Consulta do usuário:** duas palavras de 4 letras (origem e destino), digitadas em um laço interativo ou passadas na linha de comando. Maiúsculas e minúsculas na digitação são indiferentes.

- **Tratamento da leitura:** o programa remove CR, LF e espaços das pontas, ignora um eventual BOM UTF-8 e linhas em branco, e descarta (contando no relatório) linhas que não tenham exatamente 4 letras. Linhas muito longas são descartadas sem estourar o buffer.

3.2 Modelagem

O grafo é representado por **listas de adjacência**. Cada vértice guarda a palavra (`chave`, a forma usada nas comparações), a grafia original (`exibicao`), o grau e o vetor de arestas; cada aresta guarda o vértice de destino e o peso. A escolha por listas se justifica pela esparsidade: com 1590 vértices e 7249 arestas, o grau médio é 9,12 e a densidade é de apenas 0,57%. Uma matriz de adjacência teria cerca de 2,5 milhões de células para guardar poucas ligações.

Peso das arestas. O enunciado pede um grafo ponderado, mas não define o peso. Adotamos a **distância alfabética entre as duas letras que diferem** (por exemplo, a→e vale 4 e o→a vale 14). Uma diferença só de caixa (Rosa/rosa, no modo estrito) vale 1, para que todo peso seja no mínimo 1.

3.3 Processo

Passo 1 – Carga primária. O programa lê o arquivo linha a linha, valida e normaliza cada palavra, ordena todas com `qsort` e elimina as repetições, mantendo a grafia da primeira ocorrência. Com os vértices em ordem alfabética, a busca de uma palavra digitada é uma busca binária, O(log V). Em seguida, para cada par de vértices (i, j) com i < j, compara-se as 4 letras; se exatamente uma difere, cria-se a aresta nas listas dos dois vértices, já com o peso. Esse passo custa $O(V^2)$.

Passo 2 – Componentes conexos. Usamos busca em largura **iterativa** (fila em vetor, sem recursão, para não correr risco de estourar a pilha). O próprio vetor da fila agrupa os vértices de cada componente. Para cada componente são calculados o tamanho, o grau máximo (palavra central) e o grau mínimo (palavra de menor conexão). Os componentes são ordenados do maior para o menor, e em caso de empate de grau as palavras empatadas são listadas em ordem alfabética. Custo: O(V + E).

Passo 3 – Simples ou multigrafo. Em vez de assumir a resposta, o programa inspeciona as listas de adjacência: ordena os destinos de cada vértice e procura destinos iguais ao próprio vértice (laços) e destinos repetidos (arestas paralelas). Também confere que a soma dos graus é igual a $2 \cdot |E|$. Pela regra do enunciado o grafo é necessariamente simples, pois uma palavra difere de si mesma em 0 posições (nunca em 1), e dois vértices distintos são ligados por no máximo uma aresta.

Passo 4 – Dijkstra. Implementado com **heap binário de mínimo** e remoção preguiçosa (entradas obsoletas do heap são ignoradas ao serem retiradas). O algoritmo mantém vetores de distância, predecessor e vértices fechados, encerra assim que o destino é fechado e reconstrói o caminho pelos predecessores. Custo: O((V + E) log V). Como o enunciado pede para comparar sequências de arestas, o Dijkstra é executado **duas vezes** sobre o mesmo grafo:

- **[A] Custo unitário** (cada aresta custa 1): obtém o menor número de transformações.
- **[B] Custo ponderado** (cada aresta custa o seu peso): obtém a menor soma de distâncias alfabéticas.

O peso total do caminho A também é calculado, para que os dois caminhos possam ser comparados nos dois critérios.

3.4 Saída

O programa imprime quatro seções:

1. **Carga primária:** arquivo, modo de comparação, linhas lidas/válidas/ignoradas, palavras repetidas fundidas, |V| e |E|.
2. **Componentes conexos:** quantidade e, para cada componente, tamanho, palavra(s) central(is) com o grau máximo e palavra(s) de menor conexão com o grau mínimo. Componentes de uma só palavra aparecem como "isolada".
3. **Simples ou multigrafo:** número de laços, de arestas múltiplas, verificação da soma dos graus e a conclusão.
4. **Caminho entre duas palavras:** componente e grau de origem e destino; as sequências [A] e [B] com o peso de cada aresta, o número de arestas e o peso total de cada uma; e uma comparação em texto. Se as palavras estiverem em componentes diferentes, ou se alguma não constar na base, o programa informa.

3.5 Decisões de projeto

- **Peso das arestas:** distância alfabética (item 3.2), pois o enunciado não define o peso.
- **Comparação de sequências:** Dijkstra com dois critérios de custo (item 3.3), uma interpretação possível de "encontre e compare as sequências".
- **Maiúsculas e minúsculas:** a base tem 14 palavras que aparecem duas vezes, só com a caixa trocada (por exemplo, Rosa/rosa, Roma/roma, Para/para). Por padrão elas são fundidas em um único vértice (1604 linhas → 1590 vértices). A opção `-s` (modo estrito) trata cada grafia como um vértice distinto (1604 vértices). Como a grafia exibida é a da primeira ocorrência, alguns caminhos mostram palavras com inicial maiúscula (por exemplo, Rosa → … → Nova).

4. Instruções de compilação

Não foi usada IDE nem arquivo de projeto. A compilação é feita pela linha de comando, com `gcc` e `make`. **Nenhuma biblioteca externa foi carregada:** o código usa apenas a biblioteca padrão de C (`stdio.h`, `stdlib.h`, `string.h`, `limits.h`) e não precisa de `-lm`. O programa foi desenvolvido e testado em Ubuntu 24.04 com gcc 13.3.

Instalação das ferramentas (se necessário):

```
sudo apt install build-essential
```

Compilação e execução:

```
make                              # gera o executável ./teg
./teg                             # relatório completo + consulta interativa
./teg entrada.txt lago cena       # consulta direta entre duas palavras
./teg -n entrada.txt lago cena    # idem, sem o relatório dos itens 1 a 3
./teg -s entrada.txt              # modo estrito (maiúsculas distintas)
./teg --ajuda                     # lista as opções
```

Compilação manual, sem `make`:

```
gcc -std=c11 -O2 -o teg src/*.c
```

O `Makefile` compila com `-std=c11 -Wall -Wextra -Wpedantic -O2`, e o código compila sem nenhum aviso. Alvos extras: `make test` (testes automatizados) e `make debug` (compila com AddressSanitizer e UBSan). No modo interativo, uma linha vazia ou a palavra `sair` encerra o programa.

Estrutura dos arquivos entregues:

```
.
├── Makefile
├── README.md
├── entrada.txt
├── src/
│   ├── main.c        # linha de comando, relatório e consulta
│   ├── grafo.h/.c    # estruturas, carga, arestas, busca
│   ├── analise.h/.c  # componentes conexos, simples x multigrafo
│   └── dijkstra.h/.c # heap binário e Dijkstra
└── tests/run_tests.sh
```

5. Descrição e análise de resultados

5.1 Carga do grafo

Das 1604 linhas da base, todas eram válidas (0 ignoradas), mas 14 eram repetições diferindo apenas na caixa, o que resulta em **1590 vértices e 7249 arestas** no modo padrão. O grau médio é 9,12 e a densidade é 0,57%. O grau mais frequente é 7 (140 palavras). A palavra de maior grau é **caia**, com 26 vizinhos (única com esse grau); 25 palavras têm grau 0 e 39 têm grau 1.

No modo estrito (`-s`) o grafo tem **1604 vértices e 7216 arestas**. O número de arestas é menor porque palavras que só diferem na caixa deixam de se fundir e, ao mesmo tempo, passam a ser comparadas letra a letra com a caixa original.

5.2 Componentes conexos

No modo padrão há **28 componentes conexos**:

- **1 componente gigante com 1561 vértices** (98,2% das palavras). Sua palavra central é **caia** (grau 26) e há 35 palavras com grau mínimo 1.
- **2 componentes com 2 vértices:** broa–proa e IMPA–impo.
- **25 componentes com 1 vértice (palavras isoladas):** azul, Baal, bits, CNPq, Dion, Eden, Egeu, emir, Esau, Etna, expo, flor, hoje, icou, Jean, noel, Ogum, pneu, psiu, siri, UFRJ, Utah, util, York e zebu. São palavras para as quais nenhuma outra da base difere em uma única letra. Chama a atenção que 12 delas são nomes próprios ou siglas.

A escolha entre os modos mostra como a definição de “mesma palavra” afeta a conectividade. No modo estrito há **40 componentes**: o gigante (1562 vértices), 3 pares (Marx–Mary, Unix–onix e broa–proa) e 36 palavras isoladas. O par IMPA–impo, por exemplo, só existe no modo padrão: com a caixa original, IMPA e impo diferem em quatro posições.

5.3 Simples ou multigrafo

O programa encontrou **0 laços e 0 arestas múltiplas**, e a soma dos graus (14498) é igual a 2·|E|. Portanto, o grafo é **simples**, como previsto pela definição: nenhuma palavra difere de si mesma em exatamente uma posição, e dois vértices distintos são ligados por, no máximo, uma aresta.

5.4 Caminhos com Dijkstra

Resultados obtidos com o programa (A = custo unitário, B = custo ponderado):

- **mata → mala** (exemplo do enunciado): 1 aresta, peso 8, nos dois critérios; mesma sequência.
- **lago → cena:** A e B têm 4 arestas e peso total 34, mas por sequências diferentes (A: lago, cago, caga, cana, cena; B: lago, lego, cego, cega, cena). Há, portanto, mais de um caminho mínimo, e o Dijkstra devolve um deles. A sequência de 5 arestas citada no enunciado é válida, mas não é a mais curta.
- **casa → vida:** A tem 4 arestas (casa, cada, ceda, veda, vida) e B tem 5 (casa, cara, dara, dira, vira, vida); ambos com peso total 42.
- **rosa → nova:** os dois têm 3 arestas, mas o peso é 29 em A e apenas 7 em B (Rosa, rota, nota, Nova).
- **amor → odio:** os dois têm 6 arestas; o peso é 48 em A e 36 em B.
- **ilha → geou** (par mais distante do componente gigante, com 14 arestas): A usa 14 arestas e peso 98; B usa 15 arestas e peso 80. É o caso em que os critérios realmente divergem: A tem uma aresta a menos, mas B pesa 18 a menos.
- **flor → lago:** não existe sequência, pois flor é uma palavra isolada. O programa informa que as palavras estão em componentes diferentes.

Interpretação. O critério unitário responde “qual o menor número de trocas de letra?”, que é a leitura clássica da transformação de palavras. O critério ponderado favorece trocas entre letras próximas no alfabeto (como casa → cara, peso 1) e por isso pode aceitar mais arestas para pagar menos. Nos pares testados, as sequências foram diferentes em 5 de 6 casos conectados, e B foi estritamente mais leve em 3 deles. Isso mostra que a escolha do critério muda a resposta, e que a comparação pedida no enunciado é informativa.

O grafo também tem a estrutura de um “mundo pequeno”: o diâmetro do componente gigante é 14 arestas (par ilha–geou, confirmado pelo programa), mas a distância média entre duas palavras conectadas é de apenas 5,4 arestas (valor obtido com um script auxiliar de conferência).

5.5 Validação e desempenho

- **Testes automatizados:** `make test` executa 40 verificações (totais do grafo, componentes, tipo do grafo, Dijkstra, entradas inválidas, modo estrito e arquivos de entrada com defeitos) com o programa compilado com AddressSanitizer e UBSan. Todas passaram, sem erro de memória nem comportamento indefinido.
- **Conferência independente:** os resultados (|V|, |E|, número e tamanho dos componentes e custos do Dijkstra) foram comparados com um script auxiliar em Python, em 400 pares de palavras sorteados (200 por modo), sem nenhuma divergência. Esse script não faz parte da entrega.

- **Sensibilidade dos testes:** introduzimos dois defeitos de propósito (aresta para palavras com 2 diferenças e Dijkstra ponderado ignorando o peso), e a suíte os detectou.
- **Entradas defeituosas:** foram testados arquivo com BOM, CRLF, linhas em branco, linhas inválidas, uma linha de 3000 caracteres e última linha sem quebra, além de arquivo inexistente e arquivo vazio (erro com código de saída 1).
- **Desempenho:** a execução completa (carga, relatório e consulta) leva cerca de 10 ms com a base fornecida.

5.6 Limitações e possíveis melhorias

- A construção das arestas compara todos os pares, $O(V^2)$. Para 1590 palavras é instantâneo, mas para bases muito maiores seria melhor agrupar as palavras por "padrão com uma letra oculta" (por exemplo, l*go), o que reduz o custo para algo próximo de $O(V \cdot k)$, com $k = 4$.
- O peso é uma escolha nossa, já que o enunciado não o define. Outras definições (por exemplo, frequência da palavra) levariam a caminhos diferentes.
- Só são aceitas letras ASCII; palavras acentuadas seriam descartadas, o que não afeta a base fornecida, que não tem acentos.
- Quando existem vários caminhos mínimos, o Dijkstra retorna apenas um (o desempate é determinístico, pelo índice do vértice).

Conclusão

O programa carrega a base, constrói o grafo de 1590 palavras com 7249 arestas, identifica o componente gigante (1561 palavras) e as 27 palavras fora dele, confirma que o grafo é simples e encontra, via Dijkstra, as sequências de transformação entre duas palavras quaisquer, comparando o menor número de trocas com a menor soma de pesos. Os resultados foram verificados por testes automatizados e por uma conferência independente.